

ECOM EXPERTS

Ecom quotes

Manual de usuario, tecnico y de desarrollo

ecom_quotes 1.0.0 · 07/09/2026

Contenido

1. Dónde está

2. Lo primero: configurar

General

Numeración

Estados

Conversión en pedido

Depuración

3. Crear un presupuesto

4. Añadir conceptos

Un producto del catálogo

Un concepto escrito a mano

Editar o borrar una línea

5. Enviárselo al cliente

6. Los otros botones

7. Lo que ve el cliente

8. Estados

9. Preguntas frecuentes

1. Árbol de ficheros

2. Base de datos

ps_ecom_quotes

ps_ecom_quotes_state + ps_ecom_quotes_state_lang

ps_ecom_quotes_product

3. Configuración y multitienda

4. Hooks

4 bis. Instalación: por qué NO se llama a PaymentModule::install()

4 ter. Autorreparación del esquema

5. Cómo se convierte un presupuesto en pedido

5 bis. Por qué el carrito se descarta y por qué no se reclama cualquier pedido

6. Totales

7. Seguridad

8. Migraciones

9. Traducciones

Dónde van los ficheros (y por qué ahí)

Cómo se mantienen

Las tres trampas de este módulo

10. Depuración

11. Pruebas y empaquetado (_pruebas/, no se distribuye)

1. La decisión que lo condiciona todo: el presupuesto no vive en un carrito

2. Los conceptos escritos a mano y el producto oculto

3. Cómo se congela el precio

4. Envío

5. Pantallas

AdminEcomQuotesController

AdminEcomQuotesStatesController

AdminEcomQuotesImportController

6. JavaScript

7 bis. Trampas de la instalación

8. Cosas que hay que recordar al tocar el módulo

9. Probar

10. Trampas que ya costaron una tarde

1. classes/EcqOrder.php — buildCart(): autocomprobación y descarte

2. classes/EcqOrder.php — applyShipping(): recibe \$expected y comprueba su updateQty

3. classes/EcqOrder.php — syncConceptProduct(): producto comprable de verdad

4. classes/EcqOrder.php — getShippingProduct(): mismos flags + isOrderable()

5. ecom_quotes.php — hookActionValidateOrder(): dejar de reclamar pedidos ajenos

6. controllers/front/myquotes.php — los errores no pueden ser avisos

Comprobaciones antes de dar esto por hecho

Lo que NO se puede hacer (ya se estudió y no sale)

7. Importador

Orígenes

Tablas que lee

Tablas que escribe

Reglas

Volcados .sql

9. Importar del módulo antiguo

Paso 1 — de dónde se leen los datos

Paso 2 — subir un volcado

Antes de importar de verdad

11. Importador

Crear presupuestos, enviárselos al cliente y convertirlos en pedidos.

Funciona en **PrestaShop 8 y 9**, y viene traducido a **castellano, inglés y francés** (los correos, también). El idioma que ve cada cliente es el suyo.

1. Dónde está

Después de instalar el módulo aparece en el menú del back-office:

Vender → **Presupuestos**, con dos pantallas:

Pantalla	Para qué
Gestor de presupuestos	Crear, editar, enviar y convertir presupuestos
Estados	Los estados por los que pasa un presupuesto

La configuración está donde la de todos los módulos: **Módulos** → **Gestor de módulos** → **Presupuestos y ofertas a cliente** → **Configurar**.

2. Lo primero: configurar

Un solo formulario con cinco pestañas y un botón «Guardar».

Arriba del todo, antes del formulario, hay un recuadro con el **estado del módulo**: los hooks, las tablas y las entradas del menú, en verde si están y en rojo si falta algo. Al abrir esta pantalla el módulo intenta repararse solo, así que lo normal es verlo todo en verde. Si algo sigue en rojo, active «Guardar registro de depuración» en la pestaña Depuración, recargue la pantalla y mándenos el fichero `.log`.

General

- **Mostrar los presupuestos en «Mi cuenta»** — añade el enlace «Mis presupuestos» en la cuenta del cliente. El cliente solo ve los presupuestos que le haya enviado.
- **Enviar correo al cliente** — manda el correo con el detalle al pulsar «Enviar al cliente». Si lo apaga, el presupuesto igual queda visible en su cuenta, pero sin aviso.
- **Permitir añadir productos del catálogo** — si lo apaga, los presupuestos solo admiten conceptos escritos a mano.
- **Validez por defecto (días)** — días que se suman a la fecha de creación para calcular hasta cuándo vale un presupuesto nuevo.

Numeración

- **Prefijo de la referencia** — por ejemplo `PRE`.
- **Dígitos del contador** — `6` da `PRE000001`.

- **Reiniciar el contador cada año** — añade el año al prefijo (`PRE2026000001`), de forma que en enero vuelve a empezar.

Estados

Aquí se dice **qué estado corresponde a cada acción**. Vienen cinco de fábrica (Creado, Enviado, Aceptado, Cancelado, Expirado) ya asignados, así que normalmente no hay que tocar nada. Si crea estados propios en la pantalla **Estados**, vuelva aquí a asignarlos.

Conversión en pedido

- **Qué ocurre al aceptar un presupuesto**
- *Llevar al cliente al proceso de compra* — el carrito se le carga con el presupuesto y pasa por la pasarela de pago habitual de la tienda. **Es la opción recomendada.**
- *Crear el pedido directamente* — el pedido se crea sin pasar por el pago, con el estado que indique debajo.
- **Estado del pedido creado directamente** — solo se usa con la segunda opción.

Depuración

Si algo no va, actívalo, repita lo que fallaba y mándenlos los ficheros `.log` que aparecen listados en esa misma pantalla.

3. Crear un presupuesto

Vender → Presupuestos → Gestor de presupuestos → Nuevo presupuesto.

Se abre la ficha con la referencia ya asignada. Rellene:

1. **Cliente** — escriba dos letras del nombre o del correo y elíjalo de la lista.
2. **Dirección** — se cargan las del cliente al elegirlo. **Es obligatoria:** es la que determina el impuesto que se aplica, y sin ella no se puede convertir en pedido.
3. **Detalle** — el nombre del presupuesto, lo que verá el cliente.
4. **Válido hasta** — se rellena solo con los días de la configuración; se puede cambiar.
5. **Moneda.**
6. **Mostrar los impuestos** — si pone «No», el presupuesto y el PDF salen sin IVA.
7. **Envío:** - *Lo calcula la tienda en el proceso de compra* — el porte sale al pagar. - *Importe fijo indicado aquí* — escriba el importe **sin impuestos** y elija su tipo. - *Envío gratuito.*
8. **Transportista propuesto** — opcional; si lo deja en «Que lo elija el cliente», el cliente escoge al pagar.
9. **Extras** — embalaje reciclado, envolver para regalo y su mensaje.
10. **Notas internas** — para usted. **No se muestran al cliente ni salen en el PDF.**

Pulse **Guardar**.

4. Añadir conceptos

Debajo, en el panel **Conceptos**, hay dos formas:

Un producto del catálogo

Escriba el nombre o la referencia, elíjalo de la lista, seleccione la combinación si la tiene, ponga cantidad, precio (viene el del catálogo, se puede cambiar) y descuento. **Añadir al presupuesto.**

Junto a la cantidad se ve el stock disponible, solo como referencia: un presupuesto se puede hacer aunque no haya existencias.

Un concepto escrito a mano

Para lo que no está en el catálogo: mano de obra, desplazamientos, un trabajo a medida. Escriba la descripción, una referencia si quiere, elija el impuesto, y ponga cantidad, precio unitario **sin impuestos** y descuento en porcentaje. **Añadir al presupuesto.**

Editar o borrar una línea

Con el lápiz y la papelera de cada fila. Los totales de la derecha se recalculan solos.

5. Enviárselo al cliente

Botón **Enviar al cliente**. Hace tres cosas:

1. El presupuesto pasa a ser **visible en «Mi cuenta»** del cliente.
2. Cambia al estado **Enviado**.
3. Si el envío de correo está activo, le manda un correo con el detalle y un enlace.

Para reenviarlo, el mismo botón.

A partir de aquí el presupuesto **sigue siendo editable** hasta que se acepte, se cancele o caduque. Si lo modifica, vuelva a pulsar «Enviar al cliente» para que el cliente reciba la versión buena.

6. Los otros botones

Botón	Qué hace
Ver el PDF	Abre el presupuesto en PDF, con la cabecera y el pie de su tienda
Duplicar	Copia cabecera y líneas en un presupuesto nuevo. Muy útil para presupuestos parecidos
Convertir en pedido	Crea el pedido usted mismo, sin esperar al cliente
Cancelar presupuesto	Lo pasa a «Cancelado». El cliente ya no puede aceptarlo
Volver al listado	

7. Lo que ve el cliente

En **Mi cuenta** → **Mis presupuestos**, una tabla con referencia, detalle, fechas, total y estado, y tres botones por presupuesto:

- **PDF** — se lo descarga.
- **Aceptar** — según su configuración, o le lleva al proceso de compra con el carrito ya montado, o le crea el pedido directamente y le lleva a la confirmación.
- **Cancelar** — lo cancela y a usted le llega un correo.

Un presupuesto **caducado** no se puede aceptar: los botones desaparecen y el estado pasa solo a «Expirado».

8. Estados

Vender → **Presupuestos** → **Estados**.

Vienen cinco de fábrica. Puede crear los suyos con **Nuevo estado**: nombre (por idioma), color de la etiqueta y orden en la lista.

Dos avisos:

- **No se puede borrar un estado que estén usando presupuestos.**
- **No se puede borrar un estado asignado a una acción** en la configuración. Cámbielo primero en la pestaña «Estados» de la configuración.

9. Preguntas frecuentes

¿Puedo poner precios sin impuestos en el presupuesto? Sí: en la ficha, «Mostrar los impuestos» → No. El PDF sale sin IVA y con un aviso al pie.

¿Cada línea puede llevar un IVA distinto? Sí. El impuesto se elige por línea, y el PDF trae el desglose por tipo.

He cambiado un presupuesto ya enviado, ¿lo ve el cliente? Sí, el cliente ve siempre la versión actual. Pero vuelva a pulsar «Enviar al cliente» para que reciba el aviso.

¿Qué pasa con los conceptos escritos a mano cuando se acepta el presupuesto? Se convierten en productos ocultos (no aparecen en el catálogo, ni en el buscador, ni en el mapa del sitio) para que el pedido y la factura salgan con el nombre, el precio y el impuesto correctos. Están en la categoría «Conceptos de presupuesto», que no es visible.

Tengo varias tiendas. La configuración es **global**: se aplica a todas por igual, y la pantalla se lo avisa. Cada presupuesto sí guarda la tienda en la que se creó.

Algo no funciona. Active «Guardar registro de depuración» en la configuración, repita lo que fallaba y mándenos los ficheros `.log` que aparecen listados en esa misma pantalla.

Ecom Quotes — Documentación técnica

Módulo de presupuestos para PrestaShop 8 y 9.

Nombre técnico	<code>ecom_quotes</code>
Clase principal	<code>Ecom_Quotes extends PaymentModule</code>
Versión	1.0.0
Autor	Ecom Experts — <code>ecomyseo@gmail.com</code>
Licencia	AFL-3.0
Compatibilidad	PrestaShop 8.0 – 9.99
Probado en	8.2.7, 9.1.1 y 9.2.0-beta.1 (instalación y pantallas, con navegador)
Idiomas	inglés (fuente), castellano y francés
Multitienda	<code>MULTISTORE_COMPATIBILITY_PARTIAL</code> — ver apartado 3
Composer / <code>vendor/</code>	no
<code>src/</code> o <code>config/services.yml</code>	no (estructura legacy)
Overrides del núcleo	ninguno

1. Árbol de ficheros

```

ecom_quotes/
├── ecom_quotes.php           Clase del módulo: instalación, hooks y configuración
├── classes/
│   ├── EcqQuote.php         ObjectModel del presupuesto
│   ├── EcqLine.php          ObjectModel de la línea
│   ├── EcqState.php         ObjectModel del estado (multiidioma)
│   ├── EcqCalc.php          Cálculo de totales (sin estado, sin carrito)
│   ├── EcqOrder.php         Presupuesto → carrito → pedido
│   ├── EcqMail.php          Correos
│   ├── EcqImport.php        Importador (SOLO compilación de cliente)
│   ├── EcqSqlDump.php       Lector de volcados .sql (SOLO cliente)
│   ├── EcqLogger.php        Registro de depuración
│   ├── EcqSchema.php        Autorreparación del esquema
│   └── HTMLTemplateEcqQuote.php Plantilla del PDF
├── controllers/
│   ├── admin/
│   │   ├── AdminEcomQuotesController.php
│   │   ├── AdminEcomQuotesStatesController.php
│   │   └── AdminEcomQuotesImportController.php (SOLO cliente)
│   └── front/
│       └── myquotes.php
├── views/
│   ├── css/{admin.css, front.css}
│   ├── js/{admin.js, front.js}
│   └── templates/
│       ├── admin/{configure.tpl, edit.tpl, import.tpl}
│       ├── front/{myquotes.tpl, account-link.tpl}
│       ├── pdf/quote.tpl
│       └── mails/detail.tpl
├── mails/{es,en,fr}/{ecq_quote,ecq_cancel}.{html,txt}
├── sql/{install.php, uninstall.php}
├── translations/{en-US,es-ES,fr-FR}/ModulesEcomquotes{Admin,Shop}.<locale>.xlf
├── logs/                    Registro de depuración (si está activado)
└── _pruebas/                NO se distribuye – ver apartado 11

```

Todas las carpetas llevan su `index.php` con redirección.

2. Base de datos

ps_ecom_quotes

Columna	Tipo	Notas
id_ecom_quotes	INT AI PK	
reference	VARCHAR(64)	Índice. Prefijo + contador
id_customer, id_address, id_currency, id_lang, id_shop	INT	
id_state	INT	→ ecom_quotes_state
id_cart	INT	0 hasta que se acepta
id_order	INT	0 hasta que se acepta
subject	VARCHAR(255)	
notes	TEXT	Notas internas, no salen al cliente
use_tax	TINYINT	Mostrar importes con impuestos
shipping_mode	TINYINT	0 = lo calcula la tienda · 1 = importe fijo · 2 = gratuito
shipping_price	DECIMAL(20,6)	Sin impuestos
id_tax_rules_group_shipping	INT	
id_carrier	INT	Transportista propuesto
gift, gift_message, recyclable	TINYINT / TEXT / TINYINT	
show_user	TINYINT	Visible en «Mi cuenta»
salt	VARCHAR(64)	Base del token público
deleted	TINYINT	Borrado lógico
date_validity	DATE	
date_add, date_upd	DATETIME	

ps_ecom_quotes_state + ps_ecom_quotes_state_lang

id_ecom_quotes_state, color, position · name por idioma.

ps_ecom_quotes_product

Columna	Notas
<code>id_ecom_quotes</code>	→ presupuesto
<code>position</code>	Orden dentro del presupuesto
<code>line_type</code>	<code>product</code> (catálogo) o <code>concept</code> (escrito a mano)
<code>id_product</code> , <code>id_product_attribute</code>	En <code>concept</code> guarda el producto oculto generado
<code>reference</code> , <code>name</code> , <code>description</code>	
<code>unit_price</code>	DECIMAL(20,6), sin impuestos
<code>discount_percent</code>	% sobre el precio unitario
<code>quantity</code>	
<code>id_tax_rules_group</code>	
<code>tax_rate</code>	Tipo congelado al crear la línea

3. Configuración y multitienda

La decisión, explícita: la configuración es GLOBAL y los datos son POR TIENDA.

	Ámbito	Cómo
Las 15 claves <code>ECQ_*</code>	global	<code>Configuration::getGlobalValue</code> / <code>updateGlobalValue</code>
<code>ECQ_HIDDEN_CATEGORY</code>	global	una sola categoría oculta para todas las tiendas
Cada presupuesto	por tienda	columna <code>id_shop</code> , que <code>EcqQuote::add()</code> rellena con el contexto
Los estados	globales	la tabla no tiene <code>id_shop</code> ; los nombres sí son por idioma

Consecuencias que hay que respetar al tocar el módulo:

- El módulo declara `MULTISTORE_COMPATIBILITY_PARTIAL` , que es lo honesto: funciona en multitienda, pero los ajustes no se pueden diferenciar por tienda. La pantalla de configuración lo avisa cuando `Shop::isFeatureActive()` .
- Toda consulta que devuelva presupuestos pasa por `EcqQuote::shopRestriction($alias)` (el listado del back-office y `getByCustomer()` del front). En una instalación de una sola tienda esa función devuelve cadena vacía y no cambia nada.
- `EcqQuote::$id_shop` vale **0** por defecto, no 1. Con el 1 de antes, cualquier presupuesto creado sin pasar por `add()` acababa en la tienda 1 sin que se notara.

- Ojo con `new Link()` y `getPageLink()` : devuelven la URL de la tienda del contexto. Si alguna vez hace falta recorrer todas, hay que cambiar el contexto con `Shop::setContext(Shop::CONTEXT_SHOP, $id_shop)` y restaurarlo.

Clave	Por defecto	Qué hace
<code>ECQ_SHOW_IN_CUSTOMER</code>	1	Enlace «Mis presupuestos» en la cuenta
<code>ECQ_SEND_MAIL</code>	1	Enviar el correo al pulsar «Enviar al cliente»
<code>ECQ_PS_PRODUCTS</code>	1	Permitir añadir productos del catálogo
<code>ECQ_DATE_EXP_MAX</code>	30	Días de validez por defecto
<code>ECQ_PREFIX</code>	<code>PRE</code>	Prefijo de la referencia
<code>ECQ_PAD</code>	6	Dígitos del contador
<code>ECQ_RESTART_YEAR</code>	0	Añadir el año al prefijo
<code>ECQ_STATE_CREATE/SENT/ACCEPT/CANCEL/EXPIRED</code>	los 5 de instalación	Mapa acción → estado
<code>ECQ_GENERATE_ORDER</code>	0	0 = proceso de compra · 1 = pedido directo
<code>ECQ_ORDER_STATE</code>	0	Estado del pedido creado directamente
<code>ECQ_DEBUG</code>	0	Registro en <code>logs/</code>
<code>ECQ_HIDDEN_CATEGORY</code>	—	Categoría oculta de los conceptos (se crea sola)

4. Hooks

Solo tres, y los tres son necesarios:

Hook	Método	Para qué
<code>displayCustomerAccount</code>	<code>hookDisplayCustomerAccount</code>	Enlace «Mis presupuestos»
<code>actionValidateOrder</code>	<code>hookActionValidateOrder</code>	Marcar el presupuesto aceptado y guardar <code>id_order</code>
<code>actionAdminControllerSetMedia</code>	<code>hookActionAdminControllerSetMedia</code>	CSS y JS solo en las pantallas del módulo

4 bis. Instalación: por qué NO se llama a `PaymentModule::install()`

El módulo hereda de `PaymentModule` **solo** para poder usar `validateOrder()`. No es un medio de pago: no registra `paymentOptions` y no necesita las tablas de restricción por moneda, país ni transportista.

`PaymentModule::install()` llama a `addCheckboxCarrierRestrictionsForModule()`, que hace un `INSERT sin IGNORE` en `ps_module_carrier`, cuya clave primaria es `(id_module, id_shop, id_reference)`. En cualquier tienda con **dos transportistas que comparten `id_reference`** —lo normal cuando se han borrado o duplicado— el segundo insert revienta con *Duplicate entry '-1-0' for key PRIMARY* y tumba la instalación entera.

Por eso `install()` llama a `Module::install()`, saltándose al padre. En PHP 8 esa llamada al «abuelo» conserva `$this`, igual que `parent::`. Y `cleanPaymentRelations()` borra las filas de esas cuatro tablas por si un intento anterior las dejó.

4 ter. Autorreparación del esquema

Si una instalación se queda a medias, el módulo consta instalado y sus tablas no existen: cualquier pantalla que las consulte tumba el back-office con *Base table or view not found*, y el comerciante no puede ni entrar a desinstalarlo.

`EcqSchema::ensureTables()` comprueba las cuatro tablas y, si falta alguna, ejecuta `sql/install.php` (todo `CREATE TABLE IF NOT EXISTS`, así que no destruye nada). Se invoca:

- al principio de `addnewfeatures()`, que corre en `install()` y al abrir la configuración;
- en el **constructor de los tres controladores de administración**.

Además `EcqState` devuelve vacío en lugar de dejar salir la excepción, para que una tabla ausente nunca deje el back-office en blanco.

5. Cómo se convierte un presupuesto en pedido

Esta es la parte que en el módulo antiguo exigía parchear `Cart` y `PaymentModule`.

1. `EcqOrder::buildCart()` crea un `Cart` con el cliente, la dirección, la moneda y el idioma del presupuesto.
2. Por cada línea: - **producto del catálogo** → se usa tal cual; - **concepto escrito a mano** → `syncConceptProduct()` crea (o actualiza) un producto con `visibility = 'none'`, `available_for_order = 1`, `show_price = **1**`, `is_virtual = **0**`, `weight = 0`, `indexed = 0`, `out_of_stock = 1` y stock 999999, dentro de la categoría oculta `ECQ_HIDDEN_CATEGORY`. Su `id_product` se guarda en la línea, y antes de seguir se comprueba con `isOrderable()` que ha quedado **comprable en esa tienda** (`active` y `available_for_order` viven en `product_shop`: si esa fila no se escribe, el producto existe pero no entra en ningún carrito).

Los dos valores en negrita son los del arreglo del 2026-09-01, y no son cosméticos: con `show_price = 0` la línea salía **sin precio** en el carrito y el cliente la borraba; con `is_virtual = 1` el carrito era virtual, PrestaShop **se saltaba el paso del transportista** y un presupuesto con envío calculado por

la tienda salía con porte 0. 3. `freezePrice()` inserta un `SpecificPrice` **atado a `id_cart`** con el precio unitario y el descuento de la línea. Es el mismo mecanismo que usa el back-office de PrestaShop para crear pedidos con precios editados, así que el pedido, la factura y los abonos salen coherentes sin tocar el núcleo. 4. `Cart::updateQty()` añade la línea. 5. `applyShipping()` : - transportista propuesto → `Cart::setDeliveryOption()` ; - `SHIPPING_FIXED` → el porte entra como una línea más (producto oculto `ECQ-SHIPPING-<grupo de impuestos>`) y se aplica una regla de carrito de envío gratuito para que el transportista no cobre dos veces; - `SHIPPING_FREE` → solo la regla de carrito. 6. Según `ECQ_GENERATE_ORDER` : - **0** → el carrito pasa a ser el del cliente y se redirige a `order` ; - **1** → `EcqOrder::createOrder()` llama a `validateOrder()` (por eso la clase hereda de `PaymentModule` ; el módulo **no** se ofrece como medio de pago, no registra `paymentOptions`). 7. `hookActionValidateOrder` cierra el círculo: guarda `id_order` y pasa el presupuesto al estado «aceptado» — **pero solo si el pedido es de verdad de ese presupuesto** (ver abajo).

5 bis. Por qué el carrito se descarta y por qué no se reclama cualquier pedido

Este apartado no es teoría: es un incidente de producción del 2026-09-01 en el módulo hermano. Un cliente aceptó un presupuesto de 199,26 €, el producto del concepto se quedó **fuera** del carrito, `buildCart()` devolvió el carrito igual, y ocho minutos después el cliente compró otra cosa por 60,50 € desde ese mismo carrito. El módulo dio el presupuesto por «Aceptado» contra ese pedido ajeno y lo dejó bloqueado.

Tres piezas encadenadas, y las tres están cerradas:

1. **Un carrito a medio montar no se entrega.** `buildCart()` va anotando en `$expected` lo que el carrito tiene que acabar llevando, y al terminar `missingFromCart()` lo compara contra `cart_product`. Si falta algo —o si `updateQty()` devolvió falso, o si el producto del concepto no quedó comprable— `buildCart()` **devuelve `null`** y `discardCart()` borra el carrito, sus reglas de carrito `ECQ*` y sus `specific_price`. El presupuesto no se queda con `id_cart`.
2. **El carrito del presupuesto es el carrito de sesión del cliente.** Eso no se puede evitar: es lo que permite que pague. Si no paga en ese momento, PrestaShop se lo devuelve la próxima vez que entra, y lo que compre después sale de ese mismo carrito.
3. **Por eso `hookActionValidateOrder` no reclama un pedido solo por `id_cart`.** Antes de marcar nada comprueba que el presupuesto **no tenía ya pedido**, que el pedido es **del mismo cliente**, y que el pedido **contiene las líneas del presupuesto** (`orderMatchesQuote()`). Si algo no cuadra, `releaseQuoteCart()` suelta el `id_cart`, lo deja escrito en el registro del módulo y en `PrestaShopLogger`, y el presupuesto vuelve a quedar libre y editable.

Se comprueba con `_pruebas/prueba_carrito.php`, que además de todo lo anterior verifica que el total del carrito coincide con el del presupuesto **al céntimo**.

6. Totales

`EcqCalc` es una clase **sin estado y sin dependencia del carrito**. Recibe las líneas y el presupuesto y devuelve los importes. La usan el listado, el editor, el PDF y el correo, así que no puede haber dos cifras distintas para el mismo presupuesto.

Por línea: `total_excl = (precio - precio × dto/100) × cantidad`, y el impuesto sobre ese importe. El desglose por tipo impositivo sale en `tax_details`.

7. Seguridad

- **Token del cliente:** `md5(_COOKIE_KEY_ . 'ecq' . salt)`, comparado con `hash_equals`. Además se comprueba que el presupuesto **es de ese cliente** y que está marcado como visible. Un token robado de otro cliente no sirve.
 - **Back-office:** todo pasa por `ModuleAdminController`, con el token de PrestaShop y los permisos por perfil. No hay ningún `ajax.php` accesible desde fuera del enrutador.
 - **SQL:** `pSQL()` / `bqSQL()` en todas las consultas manuales; los enteros con `(int)`.
 - **Volcados .sql:** del fichero subido solo se ejecutan `CREATE TABLE` e `INSERT` de una lista blanca de tablas, renombradas a `ps_ecqsrc_`. Cualquier otra sentencia se descarta sin ejecutar.
 - **AJAX:** cabeceras `Cache-Control: no-store` y `fetch(..., {cache: 'no-store'})`.
-

8. Migraciones

No hay carpeta `upgrade/`. `addnewfeatures()` se llama en `install()` y cada vez que se abre la configuración, y aplica:

- `ensureColumn()` para cada columna añadida (consulta `INFORMATION_SCHEMA` antes del `ALTER TABLE`);
 - creación de los estados por defecto si la tabla está vacía;
 - `installTabs()` + `fixTabIntegrity()` (elimina pestañas duplicadas, que rompen `getAdminLink`);
 - `EcqQuote::refreshExpired()`.
-

9. Traducciones

`isUsingNewTranslationSystem()` devuelve `true`. Dominios:

- `Modules.Ecomquotes.Admin` — back-office y PDF
- `Modules.Ecomquotes.Shop` — front office y correos

Las cadenas fuente están **en inglés**, que es la CLAVE con la que PrestaShop busca la traducción, y los `.xlf` traen el castellano y el francés. Una cadena fuente en castellano dejaría el módulo intraducible: el sistema nuevo casa por `<source>` exacto.

En PHP se usa `$this->trans('...', array(), 'Modules.Ecomquotes.Admin')` y en las plantillas `{1 s='...' d='Modules.Ecomquotes.Admin'}`. **Nunca** `mod='...'`: es el sistema antiguo y no lee los `.xlf`.

Dónde van los ficheros (y por qué ahí)

```
translations/es-ES/ModulesEcomquotesAdmin.es-ES.xlf
translations/es-ES/ModulesEcomquotesShop.es-ES.xlf
translations/en-US/... translations/fr-FR/...
```

Esa ruta —**carpeta por idioma, un fichero por dominio**— es la única que lee el traductor en ejecución:

`PrestaShopBundle\Translation\TranslatorLanguageLoader::loadModuleTranslations()` busca `modules/<módulo>/translations/<locale>/ModulesEcomquotes<Dominio>.<locale>.xlf`. Es idéntico en 8.2, 9.1 y 9.2. Un `translations/es-ES.xlf` suelto en la raíz **no se lee en ejecución**: solo lo ve el editor de traducciones del back-office.

`ModulesEcomquotes` sale de `DomainHelper::buildModuleBaseDomain('ecom_quotes')`, que pasa a minúsculas, quita los guiones bajos y pone la inicial en mayúscula.

Cómo se mantienen

La fuente de verdad es `_pruebas/traduccion.es.tsv`: una línea por cadena, con el dominio, la clave inglesa y la traducción a cada idioma. Los `.xlf` se generan:

```
python _pruebas/generar_traduccion.py # regenera los seis ficheros
python _pruebas/revisar_traduccion.py # tiene que salir TOTAL: 0
```

El generador aborta si el TSV y el código no cuadran exactamente, así que una cadena nueva sin traducir no puede pasar desapercibida.

Las tres trampas de este módulo

1. **Cadenas dentro del JavaScript.** El extractor solo mira `.php` y `.tpl`, así que una cadena escrita en `views/js/admin.js` se queda sin traducir para siempre y nadie se entera. Las diez que había se declaran en `Ecom_Quotes::getJsTranslations()` y viajan con `Media::addJsDef` como `window.ecqI18n`.
2. **Lo que se guarda traducido en la base de datos.** Los nombres de las pestañas del menú y de los estados por defecto no pasan por el traductor en cada pantalla: se resuelven UNA vez, al instalar, en el idioma de cada tienda, con `Ecom_Quotes::translateFor()`. Como se llaman con una variable, el extractor no las ve: están declaradas con el literal a la vista en `declareDynamicStrings()`.
3. **El asunto de los correos y la cabecera de su tabla.** El asunto no está en la plantilla del correo, y el bloque del detalle se monta con `smarty->fetch()`, que traduce con el idioma del CONTEXTO —el del empleado que pulsa «Enviar al cliente»—. Los dos se resuelven con el idioma del presupuesto en `EcqMail::subject()` y `EcqMail::labels()`.

Los correos (`mails/es`, `mails/en`, `mails/fr`) **no** usan el sistema de traducciones: se duplican por carpeta de idioma. Para añadir un idioma, se copia una carpeta con su código ISO y se traduce.

10. Depuración

Con `ECQ_DEBUG` activo se escribe un fichero por día en `logs/ecom_quotes-AAAA-MM-DD.log` dentro de la carpeta del módulo. Canales: `admin`, `front`, `order`, `mail`, `pdf`, `import`. La pantalla de configuración lista los ficheros y permite borrarlos.

11. Pruebas y empaquetado (`_pruebas/`, no se distribuye)

```
python _pruebas/revisar_traduccion.py   catálogos completos y bien enrutados
python _pruebas/revisar_addons.py       lo que rechaza Addons, la parte determinista
python _pruebas/revisar_api_ps9.py      que las APIs del núcleo existan en 8.2, 9.1 y 9.2
python _pruebas/generar_traduccion.py   regenera los .xlf desde traducciones.tsv
python _pruebas/prueba_pantallas.py     navegador de verdad, pulsa todos los botones
copiar _pruebas/prueba_carrito.php      de una tienda y ejecutarlo con su PHP:
                                         el carrito del presupuesto, al centimo
python _pruebas/empaquetar.py           ZIP de venta (sin importador)
python _pruebas/empaquetar.py --cliente ZIP con el importador
```

`prueba_pantallas.py` entra al back-office, recorre las cinco pestañas de la configuración y la guarda, crea un presupuesto, le asigna cliente y dirección, le añade un concepto a mano y un producto del catálogo, **comprueba la aritmética contra la cuenta hecha en el propio guion** (2×100 con $10\% = 180$ sin impuestos), descarga el PDF y comprueba que empieza por `%PDF`, abre «Mis presupuestos» como cliente, y pulsa los botones destructivos rechazando el aviso. Exige **cero errores de consola** y deja una captura de cada estado en `_pruebas/capturas/<versión>/`.

Antes de la primera pasada, en cada tienda de pruebas:

```
copiar _pruebas/preparar_tienda.php dentro de la tienda y ejecutarlo
```

Le pone al cliente de demostración una contraseña conocida; sin eso la parte del front no se prueba, porque la pantalla pide sesión.

`empaquetar.py` aborta si algún PHP no pasa `php -l`, si falta un `index.php`, si un `.xlf` no es XML válido o si queda una sola mención al módulo de origen.

Ecom Quotes — Guía para desarrollo

Cómo está montado por dentro, qué decisiones se tomaron y por qué, y dónde tocar para cambiar cada cosa.

1. La decisión que lo condiciona todo: el presupuesto no vive en un carrito

La forma clásica de hacer esto en PrestaShop 1.4/1.5 era guardar cada concepto del presupuesto como una **personalización** de un producto oculto, dentro de un **carrito de PrestaShop vivo** desde el minuto uno. Eso obligaba a parchear cuatro clases del núcleo para que los totales, el pedido y la factura salieran bien.

Aquí el presupuesto es **datos propios**:

```
ps_ecom_quotes      cabecera
ps_ecom_quotes_product una fila por línea
```

Los totales los calcula `EcqCalc`, una clase estática sin dependencias. El carrito se crea **solo al aceptar** y se le congelan los precios. Por eso el módulo tiene **cero overrides**.

Si algún día hay que cambiar la fórmula de un total, se toca **un único sitio**: `EcqCalc::line()` y `EcqCalc::totals()`.

2. Los conceptos escritos a mano y el producto oculto

Un concepto («Mano de obra de instalación, 3 × 100 €, -10 %, IVA 21 %») no es un producto del catálogo, pero el pedido y la factura de PrestaShop **solo saben de productos**.

Solución: al aceptar el presupuesto, `EcqOrder::syncConceptProduct()` crea un producto con:

```
$product->visibility      = 'none'; // ni catálogo, ni buscador, ni sitemap
$product->available_for_order = 1; // pero sí se puede pedir
$product->show_price      = 1; // o la línea sale SIN PRECIO en el carrito
$product->is_virtual      = 0; // o el carrito es virtual y no pide transportista
$product->weight          = 0;
$product->indexed         = 0;
$product->out_of_stock    = 1; // se puede pedir sin stock
$product->id_shop_default = (int) $quote->id_shop;
$product->id_category_default = ECQ_HIDDEN_CATEGORY;
```

Y después de guardarlo, `isOrderable()`: se relee de la base de datos y se comprueba que `active` y `available_for_order` han quedado puestos **en esa tienda**. Viven en `product_shop`, y si esa fila no se escribe

el producto existe pero no entra en ningún carrito — que es justo el fallo que dejaba el carrito vacío sin que nadie se enterara.

`visibility = 'none'` es un valor **oficial** de PrestaShop (`Product::$definition['visibility']['values']`), pensado exactamente para esto. No es un truco.

El `id_product` generado se guarda en la línea, así que reconstruir el carrito no crea productos duplicados.

Si hay que cambiar algo del producto oculto, se toca `syncConceptProduct()`. Ojo: también existe `getShippingProduct()`, que es el equivalente para los gastos de envío, compartido por grupo de impuestos.

3. Cómo se congela el precio

```
$specific = new SpecificPrice();
$specific->id_cart = (int) $cart->id; // <- la clave
$specific->id_product = (int) $idProduct;
$specific->price = (float) $line['unit_price'];
$specific->reduction = (float) $line['discount_percent'] / 100;
$specific->reduction_type = 'percentage';
```

`specific_price.id_cart` es el mecanismo que usa el propio back-office cuando creas un pedido a mano y editas un precio. Al estar atado al carrito no afecta a la tienda, y el pedido, la factura y los abonos salen coherentes.

El carrito no se entrega a medias. `buildCart()` apunta en `$expected` lo que el carrito tiene que acabar llevando y al final lo compara contra `cart_product` con `missingFromCart()`. Si falta una línea, o `updateQty()` devolvió falso, o el producto del concepto no quedó comprable en esa tienda, **devuelve `null`** y `discardCart()` borra el carrito, sus reglas `ECQ*` y sus `specific_price`. Antes devolvía el carrito igual, y de ahí salió el incidente de producción del 2026-09-01: ver `DOC-tecnico.md` §5 bis y `ARREGLO-carrito-secuestrado.md`.

Limitación conocida: solo cabe un `SpecificPrice` por `(id_product, id_product_attribute, id_cart)`. Por eso dos líneas con el mismo producto y la misma combinación en un mismo presupuesto se pisarían. Los conceptos escritos a mano no tienen ese problema (cada uno lleva su producto oculto). Si se quiere permitir el mismo producto dos veces con precios distintos, la salida sería duplicar la línea como concepto.

4. Envío

`shipping_mode` tiene tres valores:

Valor	Constante	Qué hace
0	SHIPPING_NONE	No se toca nada; el porte lo calcula la tienda en el proceso de compra
1	SHIPPING_FIXED	El porte entra como una línea más (producto oculto <code>ECQ-SHIPPING-<grupo></code>) y se aplica una regla de carrito de envío gratuito, para que el transportista no vuelva a cobrarlo
2	SHIPPING_FREE	Solo la regla de carrito

La regla de carrito se crea restringida al cliente (`id_customer`), con `quantity = 1` y `quantity_per_user = 1`, así que no se puede reutilizar en otro pedido.

5. Pantallas

AdminEcomQuotesController

Hace dos cosas:

- **Listado** — `HelperList` nativo. Con `explicitSelect = true`, **toda columna propia necesita su `filter_key`** (`'a!date_add'`), o PrestaShop la selecciona sin prefijo y choca con la columna del mismo nombre de `ps_customer` (`date_add`) → *Column is ambiguous*. Ya pasó una vez; no se repita.
- **Editor** — `renderForm()` devuelve `views/templates/admin/edit.tpl`. La cabecera se guarda con un POST normal (`submitEcqHeader`); las líneas, por AJAX.

Acciones por URL, todas en `postProcess()`:

Parámetro	Qué hace
<code>newquote=1</code>	Crea un borrador y abre el editor
<code>viewpdf=<id></code>	Devuelve el PDF
<code>sendquote=<id></code>	Marca visible, pasa a «enviado» y manda el correo
<code>cancelquote=<id></code>	Pasa a «cancelado»
<code>makeorder=<id></code>	Crea el pedido
<code>duplicatequote=<id></code>	Copia cabecera y líneas en un presupuesto nuevo

Acciones AJAX (`ajaxProcessXxx`): `SearchCustomers`, `GetAddresses`, `SearchProducts`, `AddLine`, `UpdateLine`, `DeleteLine`, `GetTotals`. Todas responden JSON con `Cache-Control: no-store`.

AdminEcomQuotesStatesController

CRUD estándar sobre `EcqState`. No deja borrar un estado que esté en uso ni uno asignado a una acción en la configuración.

AdminEcomQuotesImportController

Ver el apartado 7.

6. JavaScript

`views/js/admin.js`, sin jQuery. **Todo se engancha por delegación sobre `document`**:

```
document.addEventListener('click', function (event) {
    var target = event.target.closest('a, button');
    ...
});
```

No hay ni un `document.getElementById('x').onclick = ...` sobre un botón que se pinte después. Las filas de la tabla de líneas se repintan por AJAX, así que un manejador directo dejaría de funcionar tras la primera operación — y peor: si el elemento aún no existe al cargar, `null.onclick` lanza `Cannot set properties of null` y **corta el resto del fichero**, dejando sin conectar todo lo que venga detrás.

Todo lo que se inyecta en el DOM pasa por `escapeHtml()`.

Ni una cadena de texto escrita aquí dentro. El extractor de traducciones solo mira `.php` y `.tpl`, así que una cadena en el `.js` se queda sin traducir para siempre y no salta ninguna alarma. Los textos llegan en `window.ecqI18n`, que inyecta `Ecom_Quotes::getJsTranslations()` con `Media::addJsDef`, y se leen con `t('clave')`:

```
var RESPALDO = { no_results: 'No results', ... }; // por si no llega el diccionario
function t(clave) {
    var d = window.ecqI18n || {};
    return typeof d[clave] === 'string' ? d[clave] : (RESPALDO[clave] || clave);
}
```

Para añadir una cadena: la clave en `RESPALDO` (con su inglés), la línea correspondiente en `getJsTranslations()`, y la traducción en `_pruebas/traduccion.es.tsv`.

7 bis. Trampas de la instalación

1. **No llamar a `PaymentModule::install()`**. Ver `DOC-tecnico.md`, apartado 4 bis. Se usa `Module::install()`.
2. **El esquema se autorrepara** (`EcqSchema`). Cualquier pantalla nueva debe invocar `EcqSchema::ensureTables()` en su constructor.

3. **El importador conecta con `$connect = true`**. El quinto parámetro de `Db` es justamente eso: con `false` se construye el objeto pero no se conecta, y la primera consulta muere con *Call to a member function query() on null*. Y `executeS()` devuelve `false` en vez de lanzar, así que la comprobación de conexión mira que la consulta de prueba traiga una fila.
4. **El JavaScript no se engancha solo a `DOMContentLoaded`**. El back-office puede inyectar el fichero cuando el evento ya ha saltado; entonces el manejador no corre nunca y los buscadores quedan muertos sin dar error. Se mira `document.readyState`.

8. Cosas que hay que recordar al tocar el módulo

1. `explicitSelect` + `filter_key`. Ver apartado 5.
2. **Todo campo del `HelperForm` necesita valor en `getConfigFormValues()`**, o en modo dev Smarty lanza `Undefined array key` y la configuración da 500.
3. `Tab::getIdFromClassName` **lanza `NonUniqueResult`** si hay pestañas duplicadas. Por eso el módulo usa `getTabId()` con `Db::getValue` y tiene `fixTabIntegrity()`.
4. **El menú del back-office solo pinta tres niveles**. La pestaña padre `AdminEcomQuotes` cuelga de `SELL`, no de `AdminParentOrders`: si cuelga de «Pedidos», los hijos quedan a cuarto nivel y **no se ven**.
5. `{displayPrice ...}` **no existe** en PrestaShop 8. Los precios se formatean en PHP (`EcqCalc::format()`) y a la plantilla llega la cadena ya hecha.
6. **Las llamadas estáticas desde Smarty están capadas** por la política de seguridad. Todo lo que necesite la plantilla se calcula antes y se pasa con `assign()`.
7. **Nada de `mod='...'` en las plantillas**. Con el sistema nuevo de traducciones hay que usar `d='Modules.Ecomquotes.Admin'`, o el `.xlf` se ignora.
8. **Ortografía**. Antes y después de tocar textos: `python C:\dev-tools\ortografia\revisar_ortografia.py <carpeta>` debe dar `TOTAL: 0`.

9. Probar

```
cd C:\dev-tools\testshop
python instalar_en_locales.py ecom_quotes      instala en las tres tiendas limpias

cd <la carpeta del modulo>
python _pruebas/revisar_traduccion.es.py      catalogos completos      -> TOTAL: 0
python _pruebas/prueba_pantallas.py           navegador de verdad      -> FALLAN: 0
python _pruebas/empaquetar.py                 el ZIP que se vende
```

`prueba_pantallas.py` entra al back-office, recorre las cinco pestañas de la configuración y la guarda, crea un presupuesto, le pone cliente y dirección, le añade un concepto a mano y un producto del catálogo, **comprueba los totales contra la cuenta hecha en el propio guion**, descarga el PDF, abre «Mis presupuestos» como cliente y pulsa los botones destructivos rechazando el aviso. Exige **cero errores de consola** y guarda una captura de cada estado en `_pruebas/capturas/<versión>/`.

`php -l` no basta: un error de ejecución tiene la sintaxis perfecta. De hecho el fallo de PrestaShop 9.2 que se arregló en esta entrega —un `(array) false` que tumbaba el buscador de productos con un 500— **solo se vio porque el guion mira el cuerpo de las respuestas 5xx**: el JavaScript se tragaba el error y la lista se quedaba vacía sin decir nada.

10. Trampas que ya costaron una tarde

1. **`(array) $x` NO es «un array o vacío».** Si `$x` vale `false` —y `Db::executeS()`, `Product::searchByName()`, `Customer::getAddresses()` y `glob()` devuelven `false` cuando no hay nada— `(array) false` es `array(0 => false)`. El `foreach` entra una vez, la primera clave que se lea revienta con «Trying to access array offset on value of type bool» y la petición muere con un **500**. En una llamada AJAX ese 500 no se ve: el JavaScript se lo traga y la lista se queda vacía. Se escribe siempre `is_array($x) ? $x : array()`. Pasó de verdad en PrestaShop 9.2, en el buscador de productos del editor.
2. **En PrestaShop 9 no existe `AdminModulesController`.** La configuración de un módulo ya no es `?controller=AdminModules&configure=<módulo>`, sino la ruta de Symfony `/improve/modules/manage/action/configure/<módulo>`. Cualquier guion o enlace que fabrique la URL antigua lleva a `/login` sin decir por qué.
3. **Un carrito que se le pasa al cliente es su carrito de sesión.** Si no paga en ese momento, PrestaShop se lo devuelve la próxima vez que entra y **todo lo que compre después sale de ese mismo carrito**. Por eso `hookActionValidateOrder` no reclama un pedido solo por `id_cart`: comprueba el cliente, que el presupuesto no tuviera ya pedido, y que el pedido lleve de verdad las líneas del presupuesto. Sin eso, el módulo marcaba el presupuesto como aceptado contra la compra que el cliente hizo ocho minutos más tarde. Y un producto oculto con `is_virtual = 1` hace el carrito virtual: PrestaShop se salta el transportista y el porte sale 0.
4. **PrestaShop 9 se llevó métodos del núcleo por delante.** `StockAvailable::setProductDependsOnStock()` **no existe en PS 9** (se fue con la gestión avanzada de stock), y llamarlo allí no es un aviso: es un **fatal** que se lleva todo el «aceptar presupuesto». `php -l` no lo ve y en PS 8 la pantalla funciona perfectamente. Se protege con `is_callable(array('Clase', 'metodo'))`, y `python _pruebas/revisar_api_ps9.py` compara **todas** las llamadas estáticas del módulo contra el código de 8.2.7, 9.1.1 y 9.2.0.
5. **El registrador de pestañas del núcleo va por su cuenta.** `ModuleTabRegister` recorre `controllers/admin/` y da de alta TODO controlador que no tenga ya pestaña, y lo hace DESPUÉS de `install()`. Saltarse una en `installTabs()` no basta: la crea igual, sin padre y sin nombre. Por eso `fixTabIntegrity()` borra la del importador cuando `IMPORTER` es `false`.

ecom_quotes — arreglo del carrito secuestrado (portado desde superbudgets82)

Contexto. En el módulo hermano `superbudgets82` se dio en producción (2026-09-01) este fallo: un cliente aceptó un presupuesto de 199,26 €, el módulo le construyó el carrito, el producto del concepto acabó fuera de ese carrito, el cliente compró **otra cosa** en el mismo carrito 8 minutos después y pagó 60,50 € — y el módulo marcó el presupuesto como **Aceptado** enganchado a ese pedido ajeno, dejándolo bloqueado y sin poder editarse.

Las causas son tres y `ecom_quotes` las tiene **todas**, con el código idéntico:

1. `EcqOrder::buildCart()` acumula errores por línea con `continue` y **devuelve el carrito igual**, aunque no haya podido meter nada. Nunca comprueba `cart_product`.
2. Ese carrito se queda como carrito de sesión del cliente (`myquotes.php` , `cookie->id_cart`). Si no paga en ese momento, PrestaShop se lo devuelve la próxima vez que entra y **todo lo que compre después sale de ese mismo carrito**.
3. `hookActionValidateOrder` reclama **cualquier** pedido que salga de ese carrito, solo por `id_cart` , sin comprobar cliente, ni que el presupuesto no tuviera pedido ya, ni que el pedido contenga las líneas del presupuesto.

Y dos defectos del producto oculto del concepto que empujan al cliente a borrar la línea:

- `show_price = 0` → la línea sale **sin precio** en el carrito.
- `is_virtual = 1` → el carrito es virtual, PrestaShop se salta el transportista y un presupuesto con envío calculado por la tienda sale con **porte 0**.

Los seis cambios de abajo son los que ya están aplicados y **verificados en producción** en `superbudgets82` . Aplícalos tal cual; son sustituciones de cadena exacta.

Reglas del proyecto que hay que respetar al aplicarlos: estructura legacy (nada de `src/` , `config/` , Composer), `array()` en vez de `[]` , castellano con tildes en los textos, y **nunca editar por rangos de línea** — sustitución de cadena exacta y única.

1. `classes/EcqOrder.php` — `buildCart()` : autocomprobación y descarte

Buscar (desde `$previousCart = ...` hasta el cierre del método, líneas ~84-136):

```

$previousCart = Context::getContext()->cart;
Context::getContext()->cart = $cart;

foreach ($lines as $line) {
    $idProduct = (int) $line['id_product'];

    if ($line['line_type'] === EcqLine::TYPE_CONCEPT) {
        $idProduct = self::syncConceptProduct($line, $quote);
        if (!$idProduct) {
            $errors[] = 'No se ha podido preparar el concepto "' . $line['name'] . '".';
            continue;
        }
    }

    if (!$idProduct) {
        $errors[] = 'Línea sin producto asociado: ' . $line['name'];
        continue;
    }

    self::freezePrice($cart, $line, $idProduct);

    $added = $cart->updateQty(
        (int) $line['quantity'],
        $idProduct,
        (int) $line['id_product_attribute'],
        false,
        'up',
        (int) $address->id,
        null,
        false
    );

    if ($added !== true && $added <= 0) {
        $errors[] = 'No se ha podido añadir al carrito: ' . $line['name'];
    }
}

self::applyShipping($cart, $quote, $errors);

$cart->update();
$quote->id_cart = (int) $cart->id;
$quote->update();

Context::getContext()->cart = $previousCart;

EcqLogger::log(
    'Carrito ' . (int) $cart->id . ' construido para el presupuesto ' . $quote-
>reference,
    'order',
    $errors
);

return $cart;
}

```

Sustituir por:

```

$previousCart = Context::getContext()->cart;
Context::getContext()->cart = $cart;

// Lo que el carrito TIENE que llevar al terminar: "id_product-id_attribute" => cantidad.
$expected = array();

foreach ($lines as $line) {
    $idProduct = (int) $line['id_product'];

    if ($line['line_type'] === EcqLine::TYPE_CONCEPT) {
        $idProduct = self::syncConceptProduct($line, $quote);
        if (!$idProduct) {
            $errors[] = 'No se ha podido preparar el concepto "' . $line['name'] . '".';
            continue;
        }
    }

    if (!$idProduct) {
        $errors[] = 'Línea sin producto asociado: ' . $line['name'];
        continue;
    }

    self::freezePrice($cart, $line, $idProduct);

    $added = $cart->updateQty(
        (int) $line['quantity'],
        $idProduct,
        (int) $line['id_product_attribute'],
        false,
        'up',
        (int) $address->id,
        null,
        false
    );

    if ($added !== true && $added <= 0) {
        $errors[] = 'No se ha podido añadir al carrito: ' . $line['name'];
        continue;
    }

    $key = (int) $idProduct . '-' . (int) $line['id_product_attribute'];
    $expected[$key] = (isset($expected[$key]) ? $expected[$key] : 0) + (int)
$line['quantity'];
}

self::applyShipping($cart, $quote, $errors, $expected);

$cart->update();

// Autocomprobación: el carrito debe llevar EXACTAMENTE lo que dice el presupuesto.
// Si falta algo, el cliente acabaría en la caja con un carrito incompleto (o vacío)
// y cualquier cosa que comprase después quedaría enganchada a este presupuesto.
foreach (self::missingFromCart($cart, $expected) as $missing) {
    $errors[] = 'La línea no ha llegado al carrito: ' . $missing;
}

```

```

Context::getContext()->cart = $previousCart;

if ($errors) {
    EcqLogger::log(
        'Carrito descartado para el presupuesto ' . $quote->reference,
        'order',
        $errors
    );
    self::discardCart($cart);

    return null;
}

$quote->id_cart = (int) $cart->id;
$quote->update();

EcqLogger::log(
    'Carrito ' . (int) $cart->id . ' construido para el presupuesto ' . $quote-
>reference,
    'order'
);

return $cart;
}

/**
 * Comprueba que el carrito lleva de verdad lo que se esperaba.
 *
 * @param Cart $cart
 * @param array $expected "id_product-id_attribute" => cantidad
 *
 * @return array descripciones de lo que falta (vacío si está todo)
 */
protected static function missingFromCart(Cart $cart, array $expected)
{
    $missing = array();

    $rows = Db::getInstance()->executeS(
        'SELECT `id_product`, `id_product_attribute`, SUM(`quantity`) AS `quantity`
        FROM `'. _DB_PREFIX_ . `cart_product`
        WHERE `id_cart` = ' . (int) $cart->id . '
        GROUP BY `id_product`, `id_product_attribute`'
    );

    $inCart = array();
    foreach (is_array($rows) ? $rows : array() as $row) {
        $inCart[(int) $row['id_product'] . '-' . (int) $row['id_product_attribute']] = (int)
        $row['quantity'];
    }

    foreach ($expected as $key => $quantity) {
        if (!isset($inCart[$key]) || $inCart[$key] < (int) $quantity) {
            $missing[] = 'producto ' . $key . ' (cantidad ' . (int) $quantity . ')';
        }
    }
}

```

```

        return $missing;
    }

    /**
     * Tira un carrito a medio construir para que no quede vivo en la sesion del
     * cliente ni enganchado al presupuesto.
     *
     * @param Cart $cart
     */
    protected static function discardCart(Cart $cart)
    {
        if (!Validate::isLoadedObject($cart)) {
            return;
        }

        // Las reglas de carrito que ha creado el modulo para ESTE carrito.
        $rules = Db::getInstance()->executeS(
            'SELECT cr.`id_cart_rule` FROM `.` . _DB_PREFIX_ . `cart_cart_rule` ccr
            INNER JOIN `.` . _DB_PREFIX_ . `cart_rule` cr ON cr.`id_cart_rule` =
            ccr.`id_cart_rule`
            WHERE ccr.`id_cart` = ' . (int) $cart->id . ' AND cr.`code` LIKE "ECQ%"'
        );
        foreach (is_array($rules) ? $rules : array() as $rule) {
            $cartRule = new CartRule((int) $rule['id_cart_rule']);
            if (Validate::isLoadedObject($cartRule)) {
                $cartRule->delete();
            }
        }

        SpecificPrice::deleteByIdCart((int) $cart->id);

        try {
            $cart->delete();
        } catch (Exception $e) {
            EcqLogger::log('No se ha podido borrar el carrito ' . (int) $cart->id . ': ' . $e-
                >getMessage(), 'order');
        }
    }
}

```

2. classes/EcqOrder.php — applyShipping() : recibe \$expected y comprueba su updateQty

Buscar:

```

    * @param Cart      $cart
    * @param EcqQuote  $quote
    * @param array     $errors
    */
    protected static function applyShipping(Cart $cart, EcqQuote $quote, array &$errors)
    {

```

(el bloque de `@param` puede tener otra alineación; lo que importa es la firma). **Sustituir la firma por:**

```
protected static function applyShipping(Cart $cart, EcqQuote $quote, array &$errors, array
&$expected = array())
{
```

y añadir en el docblock:

```
* @param array    $expected lo que el carrito tiene que acabar llevando
```

Buscar (dentro de `applyShipping`, rama `SHIPPING_FIXED`):

```
self::freezePrice($cart, $line, $idProduct);
$cart->updateQty(1, $idProduct, 0, false, 'up', (int) $cart->id_address_delivery,
null, false);
} elseif ($mode !== EcqQuote::SHIPPING_FREE) {
```

Sustituir por:

```
self::freezePrice($cart, $line, $idProduct);
$dadded = $cart->updateQty(1, $idProduct, 0, false, 'up', (int) $cart-
>id_address_delivery, null, false);

if ($dadded !== true && $dadded <= 0) {
    $errors[] = 'No se ha podido añadir al carrito el concepto de envío.';

    return;
}

$key = (int) $idProduct . '-0';
$expected[$key] = (isset($expected[$key]) ? $expected[$key] : 0) + 1;
} elseif ($mode !== EcqQuote::SHIPPING_FREE) {
```

3. `classes/EcqOrder.php` — `syncConceptProduct()` : producto comprable de verdad

Buscar:

```
$product->reference = 'ECQ-' . (int) $line['id_ecom_quotes_product'];
$product->price = (float) $line['unit_price'];
$product->id_tax_rules_group = (int) $line['id_tax_rules_group'];
$product->active = 1;
$product->visibility = 'none';
$product->available_for_order = 1;
$product->show_price = 0;
$product->indexed = 0;
$product->is_virtual = 1;
$product->minimal_quantity = 1;
$product->out_of_stock = 1;
$product->id_category_default = self::getHiddenCategory();
$product->redirect_type = '404';

$saved = $idProduct ? $product->update() : $product->add();
if (!$saved) {
    return 0;
}

$product->addToCategories(array(self::getHiddenCategory()));
StockAvailable::setProductDependsOnStock((int) $product->id, false);
StockAvailable::setProductOutOfStock((int) $product->id, 1);
StockAvailable::setQuantity((int) $product->id, 0, 999999, (int) $quote->id_shop);

if ((int) $line['id_product'] !== (int) $product->id) {
```

Sustituir por:

```

$product->reference = 'ECQ-' . (int) $line['id_ecom_quotes_product'];
$product->price = (float) $line['unit_price'];
$product->id_tax_rules_group = (int) $line['id_tax_rules_group'];
$product->active = 1;
$product->visibility = 'none';
$product->available_for_order = 1;
// show_price a 1: con 0 la linea sale SIN precio en el carrito y el cliente
// no reconoce lo que esta comprando (y la borra).
$product->show_price = 1;
$product->indexed = 0;
// is_virtual a 0: un carrito virtual no pide transportista y se salta el
// porte, asi que un presupuesto con envio calculado por la tienda saldria a 0.
$product->is_virtual = 0;
$product->weight = 0;
$product->minimal_quantity = 1;
$product->out_of_stock = 1;
$product->id_shop_default = (int) $quote->id_shop;
$product->id_category_default = self::getHiddenCategory();
$product->redirect_type = '404';

$saved = $idProduct ? $product->update() : $product->add();
if (!$saved) {
    return 0;
}

$product->addToCategories(array(self::getHiddenCategory()));
StockAvailable::setProductDependsOnStock((int) $product->id, false);
StockAvailable::setProductOutOfStock((int) $product->id, 1);
StockAvailable::setQuantity((int) $product->id, 0, 999999, (int) $quote->id_shop);

if (!$self::isOrderable((int) $product->id, (int) $quote->id_shop)) {
    EcqLogger::log(
        'El concepto ' . $product->reference . ' (producto ' . (int) $product->id . ') no
ha quedado comprable.',
        'order'
    );

    return 0;
}

if ((int) $line['id_product'] !== (int) $product->id) {

```

4. classes/EcqOrder.php — getShippingProduct() : mismos flags + isOrderable()

Buscar:

```
$product->reference = $reference;
$product->price = 0;
$product->id_tax_rules_group = (int) $idTaxRulesGroup;
$product->active = 1;
$product->visibility = 'none';
$product->available_for_order = 1;
$product->show_price = 0;
$product->indexed = 0;
$product->is_virtual = 1;
$product->minimal_quantity = 1;
$product->out_of_stock = 1;
$product->id_category_default = self::getHiddenCategory();
$product->redirect_type = '404';

$saved = $idProduct ? $product->update() : $product->add();
if (!$saved) {
    return 0;
}

$product->addToCategories(array(self::getHiddenCategory()));
StockAvailable::setProductDependsOnStock((int) $product->id, false);
StockAvailable::setProductOutOfStock((int) $product->id, 1);
StockAvailable::setQuantity((int) $product->id, 0, 999999);

return (int) $product->id;
}
```

Sustituir por:

```

        $product->reference = $reference;
        $product->price = 0;
        $product->id_tax_rules_group = (int) $idTaxRulesGroup;
        $product->active = 1;
        $product->visibility = 'none';
        $product->available_for_order = 1;
        $product->show_price = 1;
        $product->indexed = 0;
        $product->is_virtual = 0;
        $product->weight = 0;
        $product->minimal_quantity = 1;
        $product->out_of_stock = 1;
        $product->id_category_default = self::getHiddenCategory();
        $product->redirect_type = '404';

        $saved = $idProduct ? $product->update() : $product->add();
        if (!$saved) {
            return 0;
        }

        $product->addToCategories(array(self::getHiddenCategory()));
        StockAvailable::setProductDependsOnStock((int) $product->id, false);
        StockAvailable::setProductOutOfStock((int) $product->id, 1);
        StockAvailable::setQuantity((int) $product->id, 0, 999999);

        if (!$self::isOrderable((int) $product->id, (int) Context::getContext()->shop->id)) {
            EcqLogger::log(
                'El concepto de envío ' . $reference . ' no ha quedado comprable.',
                'order'
            );

            return 0;
        }

        return (int) $product->id;
    }

/**
 * Comprueba que un producto existe y se puede meter en un carrito EN ESA TIENDA.
 *
 * Se lee de nuevo de la base de datos a proposito: los campos active,
 * available_for_order y show_price viven en product_shop, y si la fila de la
 * tienda no se ha escrito el producto existe pero no se puede pedir. Es
 * exactamente el fallo que deja el carrito vacio sin que nadie se entere.
 *
 * @param int $idProduct
 * @param int $idShop
 *
 * @return bool
 */
protected static function isOrderable($idProduct, $idShop)
{
    if (!(int) $idProduct) {
        return false;
    }

```

```
$product = new Product((int) $idProduct, false, null, (int) $idShop);

if (!Validate::isLoadedObject($product)) {
    return false;
}

return (bool) $product->active && (bool) $product->available_for_order;
}
```

5. `ecom_quotes.php` — `hookActionValidateOrder()` : dejar de reclamar pedidos ajenos

Este es el cambio importante: es el que habría evitado el incidente.

Buscar:

```
$order = $params['order'];
$idQuote = EcqQuote::getIdByCart((int) $order->id_cart);
if (!$idQuote) {
    return;
}

$quote = new EcqQuote((int) $idQuote);
if (!Validate::isLoadedObject($quote)) {
    return;
}

$map = EcqState::map();
```

Sustituir por:

```

$order = $params['order'];
$idQuote = EcqQuote::getIdByCart((int) $order->id_cart);
if (!$idQuote) {
    return;
}

$quote = new EcqQuote((int) $idQuote);
if (!Validate::isLoadedObject($quote)) {
    return;
}

// El carrito de un presupuesto se queda como carrito de sesión del cliente.
// Si no paga en ese momento, PrestaShop se lo devuelve la próxima vez que
// entra y todo lo que compre después sale de ESE MISMO carrito. Sin estas
// comprobaciones el módulo daba por aceptado el presupuesto con un pedido
// que no tenía nada que ver.
if ((int) $quote->id_order) {
    $this->releaseQuoteCart($quote, $order, 'el presupuesto ya tenía pedido');

    return;
}
if ((int) $quote->id_customer !== (int) $order->id_customer) {
    $this->releaseQuoteCart($quote, $order, 'el pedido es de otro cliente');

    return;
}
if (!$this->orderMatchesQuote($order, $quote)) {
    $this->releaseQuoteCart($quote, $order, 'el pedido no contiene las líneas del
presupuesto');

    return;
}

$map = EcqState::map();

```

Y añadir los dos métodos justo después del cierre de `hookActionValidateOrder()` . Buscar:

```

EcqLogger::log(
    'Presupuesto ' . $quote->reference . ' aceptado con el pedido ' . (int) $order->id,
    'order'
);
}

```

Sustituir por:

```

        EcqLogger::log(
            'Presupuesto ' . $quote->reference . ' aceptado con el pedido ' . (int) $order->id,
            'order'
        );
    }

/**
 * ¿El pedido lleva de verdad todas las líneas del presupuesto?
 *
 * @param Order    $order
 * @param EcqQuote $quote
 *
 * @return bool
 */
protected function orderMatchesQuote($order, EcqQuote $quote)
{
    $lines = $quote->getLines();
    if (!$lines) {
        return false;
    }

    $rows = Db::getInstance()->executeS(
        'SELECT `product_id` FROM `' . _DB_PREFIX_ . 'order_detail` WHERE `id_order` = ' .
(int) $order->id
    );

    $inOrder = array();
    foreach (is_array($rows) ? $rows : array() as $row) {
        $inOrder[(int) $row['product_id']] = true;
    }

    foreach ($lines as $line) {
        $idProduct = (int) $line['id_product'];
        if (!$idProduct || !isset($inOrder[$idProduct])) {
            return false;
        }
    }

    return true;
}

/**
 * Suelta el carrito del presupuesto cuando el pedido que ha salido de él no
 * le corresponde. Así el presupuesto vuelve a quedar libre y editable en vez
 * de quedarse "Aceptado" contra un pedido ajeno.
 *
 * @param EcqQuote $quote
 * @param Order    $order
 * @param string    $motivo
 */
protected function releaseQuoteCart(EcqQuote $quote, $order, $motivo)
{
    $mensaje = 'Presupuesto ' . $quote->reference . ' NO se marca aceptado con el pedido '
        . (int) $order->id . ': ' . $motivo . '.';

```

```

        if ((int) $quote->id_cart === (int) $order->id_cart) {
            $quote->id_cart = 0;
            $quote->update();
            $mensaje .= ' Se ha soltado el carrito ' . (int) $order->id_cart . '.';
        }

        EcqLogger::log($mensaje, 'order');
        PrestaShopLogger::addLog('[ecom_quotes] ' . $mensaje, 2, null, 'Order', (int) $order->id,
true);
    }

```

6. controllers/front/myquotes.php — los errores no pueden ser avisos

Con el cambio 1, `buildCart()` ya devuelve `null` ante cualquier error, así que este bloque queda muerto y encima confunde.

Buscar:

```

        $this->errors[] = $this->trans('The cart could not be prepared.', array(),
'Modules.Ecomquotes.Shop');

        return;
    }
    foreach ($errors as $error) {
        $this->warning[] = $error;
    }

    $this->context->cookie->id_cart = (int) $cart->id;

```

Sustituir por:

```

        $this->errors[] = $this->trans('The cart could not be prepared.', array(),
'Modules.Ecomquotes.Shop');

        return;
    }

    $this->context->cookie->id_cart = (int) $cart->id;

```

Comprobaciones antes de dar esto por hecho

1. `php -l` en los tres ficheros, con PHP 7.4 y 8.3.
2. `python C:\dev-tools\ortografia\revisar_ortografia.py <carpeta del modulo> → TOTAL: 0`.
3. **Aceptar un presupuesto de verdad** con una línea de concepto libre y comprobar que el total del carrito coincide con el del presupuesto, hasta el céntimo. En [superbudgets82](#) se hizo con un guion PHP que

arranca PrestaShop (`require config/config.inc.php`), crea el presupuesto y la línea, llama a `EcqOrder::buildCart()` y vuelca `cart_product` , `cart_cart_rule` , `$cart->getOrderTotal(true, Cart::BOTH)` y la fila de `product_shop` / `stock_available` del producto del concepto.

4. **Abrirlo con el navegador** y llegar hasta el pedido pagado. Ojo al **paso del transportista**: con `is_virtual = 0` PrestaShop ya sí lo pide (antes se lo saltaba). Con envío gratuito el importe sale 0 gracias al vale, pero tiene que haber un transportista disponible para esa dirección y para un producto de 0 kg.

Lo que NO se puede hacer (ya se estudió y no sale)

Sustituir el producto oculto por línea por **un solo producto base compartido** no funciona: `specific_price` va por `(id_cart, id_product)` , así que dos conceptos con precios distintos en el mismo presupuesto no caben; y el carrito muestra el nombre del producto, con lo que todas las líneas saldrían con el mismo texto genérico y dos clientes simultáneos verían el texto del presupuesto del otro. Se mantiene un producto oculto por línea, reutilizado (la línea guarda su `id_product`) y con la garantía de que existe y es comprable.

Ecom Quotes — El importador (solo compilación de cliente)

Este documento NO va en el ZIP de venta. El importador solo se compila para el cliente que migra desde un módulo de presupuestos anterior. En la versión que se vende, `Ecom_Quotes::IMPORTER` vale `false` y `_pruebas/empaquetar.py` saca del paquete el controlador, `EcqImport`, `EcqSqlDump`, `import.tp1` y este mismo fichero.

Para generar la compilación del cliente:

```
python _pruebas/empaquetar.py --cliente
```

7. Importador

Tres clases:

- `EcqImport` — lee las tablas del módulo antiguo y escribe las nuevas.
- `EcqSqlDump` — carga un volcado `.sql` en tablas temporales.
- `AdminEcomQuotesImportController` — la pantalla.

Orígenes

Origen	Cómo
<code>local</code>	Misma base de datos, otro prefijo
<code>remote</code>	Otra base MySQL (host, base, usuario, contraseña)
<code>dump</code>	Tablas temporales cargadas desde un volcado <code>.sql</code>

Tablas que lee

`superbudgets`, `superbudgets_state` (obligatorias); `superbudgets_products`, `superbudgets_cart`, `tax`, `customization`, `customized_data`, `customization_field_lang`, `cart_product` (opcionales, las tres últimas como respaldo).

Tablas que escribe

`ecom_quotes`, `ecom_quotes_state`, `ecom_quotes_state_lang`, `ecom_quotes_product`. **Nada más.**

Reglas

- **Repetible:** si la referencia ya existe, se omite. Nunca duplica.
- **Simulación** (`setDryRun(true)`) que cuenta sin escribir.

- Los estados se reutilizan **por nombre**.
- El `id_tax` antiguo se traduce a porcentaje leyendo la tabla `tax` del origen, y se busca el grupo de reglas local con ese mismo porcentaje. Si no lo hay, el porcentaje se guarda igualmente en `tax_rate`, así que el presupuesto conserva sus importes.
- **No envía ningún correo ni dispara nada hacia el cliente.** Esto es deliberado: el importador toca datos históricos y una acción externa sobre registros antiguos es exactamente el fallo que no se puede cometer.

Volcados `.sql`

`EcqSqlDump::rewrite()` acepta **solo** dos formas de sentencia:

```
CREATE TABLE [IF NOT EXISTS] `<px><tabla de la lista blanca>` ...  
INSERT [IGNORE] INTO `<px><tabla de la lista blanca>` ...
```

Cualquier otra cosa (`DROP`, `ALTER`, `UPDATE`, `SET`, procedimientos, o un `INSERT` sobre una tabla que no esté en la lista) se descarta **sin ejecutarla**. El nombre de la tabla se sustituye **solo en su primera aparición**, que es la del propio `CREATE` / `INSERT`, para no tocar los valores insertados.

9. Importar del módulo antiguo

Vender → **Presupuestos** → **Importar**. Es para traer los presupuestos que tuviera en el módulo `superbudgets` de PrestaShop 1.4 o 1.5.

La propia pantalla lleva escrita **la lista completa de tablas que lee y de tablas que escribe**. No toca pedidos, ni clientes, ni el catálogo, y **no envía ningún correo**.

Paso 1 — de dónde se leen los datos

Antes de nada, pulse «**Probar la conexión**»: le dirá si llega al origen y qué tablas del módulo antiguo encuentra. Es la forma rápida de saber si el prefijo o los datos de la base son los correctos.

Tres opciones:

- **De esta misma base de datos, con otro prefijo** — cuando la tienda se migró y las tablas viejas siguen ahí. La pantalla le dice qué prefijos ha encontrado.
- **De otra base de datos MySQL** — servidor, base, usuario y contraseña.
- **De un volcado `.sql` ya cargado** — después de hacer el paso 2.

También hay un campo **ID del producto oculto antiguo**: es el número que tenía el ajuste `SUPERBUDGET_PRODUCT_DEFAULT` en la tienda vieja (el producto con referencia `super_budget`). Sirve para distinguir qué líneas eran conceptos escritos a mano. Si no lo sabe, déjelo en 0.

Paso 2 — subir un volcado

Elija el fichero `.sql` de la tienda antigua e indique **el prefijo que tenían las tablas ahí** (normalmente `ps_`). Del volcado solo se cargan las tablas del módulo, en tablas temporales; el resto del fichero se descarta. Cuando termine, vuelva al paso 1 y elija «De un volcado .sql ya cargado».

Al acabar, **Borrar las tablas temporales**.

Antes de importar de verdad

Pulse siempre primero «Simular»: cuenta lo que entraría sin escribir nada. Si el resultado le cuadra, pulse «Importar».

La importación **se puede repetir**: un presupuesto cuya referencia ya exista se omite, no se duplica.

11. Importador

Ver `DOC-desarrollo.md`, apartado «Importador», y la propia pantalla **Vender → Presupuestos → Importar**, que lleva escrita la lista de tablas que lee y que escribe.
